



Diamond Doc

A Software Documentation

Seepick

Version SNAPSHOT, 2025-12-07

Table of Contents

- 1. Development. 1
 - 1.1. Best Practices 1
 - 1.1.1. Coding Philosophy 1
 - 1.1.2. Conciseness 1
 - 1.1.3. Code Samples 2
 - 1.2. Local Development 2
 - 1.2.1. IntelliJ Plugins 2
 - 1.3. Code Generation 3
 - 1.3.1. Generating Sources 3
 - 1.3.2. Generate Persistence Code 4

Chapter 1. Development

1.1. Best Practices

- Strive for functional, e.g.: **pure functions**:
 - return values are identical for identical arguments (no variation with static variables, non-local variables, mutable reference arguments or input streams, i.e., referential transparency)
 - the function has no side effects (no mutation of non-local variables, mutable reference arguments or input/output streams)
- no overloaded methods, provide different name
- instead of regular methods → "named expressions"
 - functions as named codeblocks, lead to shorter (easier to comprehend) methods
 - always use "=" after method signature; single expression body
 - no state, no "val" keyword, all just immediate execution (more functional)
 - forces to write very short methods, easy to read and comprehend
 - methods have names, they create logical structure of arbitrary code

1.1.1. Coding Philosophy

- fail early, fail fast
- e.g. if application config something is configured wrong/missing
- validate data (front incoming HTTP, back external systems/DB, immediately throw)
- code first / code over config
 - no strings/annotations, no declarative xml/json/yaml/properties
 - just code, be in control, flexible, safe (compiler reference check)
- no classpath scanning, reflection, other look-ups/service-locator, no auto-magically something
- avoid intrusive frameworks (the systems serves us, we don't serve the system)
- functional programming
- pure functions: stateless, side-effect free
- code as named expressions
- prefer single-expression-method **fun foo() = doSome().doOther().also { done(it) }**
- absolutely no local files (IDE specific, build generated, ...)
 - not checked in, even existing put on .gitignore
 - all auto-configured by build system/gradle
 - don't interfere with each other; give devs freedom - no coupling
- single responsibility (of package, class, method/function)
- clear names, self-explanatory, capturing its essence (single responsibility)
 - no abbreviations, no short cryptic names

1.1.2. Conciseness

Using type inference and [single expression functions](#), function declarations can be shortened and made more readable:

Ommitting the return type

```
fun `BAD explicit`(): String = "foo"
```

```
fun `GOOD inferred`() = "foo"
```

Using lambda wrappers

```
fun `BAD increase indentation`() {  
    transactional {  
        repo.update()  
    }  
}
```

```
fun `GOOD single expression`() = transactional { ❶  
    repo.update()  
}
```

❶ Sometimes needed to override inferred type to **: Unit**.

Using standard functionals

```
fun Service.`BAD return`(): Service {  
    setFoo("bar")  
    return this  
}  
  
fun Service.`GOOD apply`() = apply {  
    setFoo("bar")  
}
```

1.1.3. Code Samples

Source of **healthRouting.kt** from the **view-routing** module

```
package nl.uwv.smz.diamond.view.routing  
  
import io.ktor.server.application.Application  
import io.ktor.server.response.respond  
import io.ktor.server.routing.get  
import io.ktor.server.routing.routing  
import nl.uwv.smz.diamond.view.controllerApi.HealthController  
import org.koin.ktor.ext.inject  
  
internal fun Application.installHealthRouting() {  
    val controller by inject<HealthController>()  
    routing {  
        get("/health") {  
            call.respond(controller.fetchHealthReport())  
        }  
    }  
}
```

1.2. Local Development

The goal is to only have to check out the sourcecode, and run a single command to start the application up in a local model. No further setup must be needed (no docker, no special files created). There is no required readme to be read, no manual or instructions; it just works out of the box.

- run **nl.uwv.smz.diamond.app.LocalDiamondApp**
- Postman collection can be found at: **/config/Diamond.postman_collection.json**
- There are a lot of useful (required) build scripts in **/bin/*.sh**

1.2.1. IntelliJ Plugins

The following plugins are recommended to have a smooth development experience:

- Gradle
- EditorConfig - autoconfigures styling
- detekt - configure with `/config/detekt.yml`
- ktlint - auto-formatter
- kotest - test framework integration
- Markdown - some internal documentation files use it
- AsciiDoc - documentation written in it
- SonarQube for IDE - connector to quality gates
 - Configure with SonarQube Cloud project: https://sonarcloud.io/project/overview?id=seepick_diamond
- Gherkin - Test specifications support
- Cucumber for Kotlin - Step definitions; regexp
- Cucumber for Java - not sure, i guess it adds to it (?)
- Cucumber+ - Better support
- Cucumber Table Mapping - Specific support
- kotlin-fill-class - Useful inline code generator

Other:

- plantuml4idea (graphviz/dot required)

1.3. Code Generation

1.3.1. Generating Sources



Status: Draft

Created: 6.12.2025

Author: Seepick

Context:

- We use specifications to document the contract, provided by ourselves and third parties.
- Primarily OpenAPI YAML and WSDL XML files.

Should we regenerate sources each time and host the code next to our own, or should be pre-generate and store them in a dedicated repository?

Options:

Re-generate:

- GOOD: Everything is self-contained; simple, easy.
- BAD: Slows down the build process due to regeneration despite no API changes (caching?).
- BAD: Exclusion configuration necessary for static code analysis, coverage, etc. (different rules apply to them)

Pre-generate:

- GOOD: Faster build times.
- GOOD: A change of the API contract must be a conscious act
 - Leads to more safety/stability
 - But it's also a BAD argument: slower development time, more complex!
- For 3rd party APIs, we would expect a given SDK anyways... so we do it ourselves.

Decision:

- Pre-generate!

- Divide and conquer: Manage the bigger complexity by outsourcing.
- Split things apart, separation of concerns.

1.3.2. Generate Persistence Code



Status: Draft

Created: 6.12.2025

Author: Seepick

Context:

- liquibase migration strategy...
- Generate Liquibase by classes; or generate classes by liquibase?
 - Back then possible with JPA
 - Or we keep it in-sync by hand
 - How is it different than API-spec?
 - maybe good to have file for contract, to make more aware if you do changes

Other (semi-related) problem: How to handle if there are MANY changelogs?

- It slows down startup considerably
- Compress/squash them?!